



Assessment Cover Sheet

Assessment Title	Project (Individual)		
Assessment Type	Uncontrolled	Individual	Not must-pass
Due Date	04-06-2026	Course Code	IT9201
Course Title	Machine Learning and Data Mining		
Internal Moderator's Name	Dr. David Gulua		
External Examiner's Name			

Instructions:

1. This cover sheet must be completed (section in blue below) and attached to your assessment before submission in hard copy/soft copy.
2. The time allowed for this assessment is until 20-05-2026 11:59 PM.
3. This assessment carries 100 marks assessing CILO 1, CILO 2 and CILO 3.
4. The **use of generative AI tools is restricted to knowledge building and data generation.**
5. References consulted (if any) must be properly acknowledged and cited.

Learner ID	202509031	Date Submitted	03-06-2026
Learner Name	Maram Alkhenaizi		
Programme Code	IT9201		
Programme Title	Master of Science in Artificial Intelligence		
Lecturer's Name	Dr. Shomona Gracia Jacob		

By submitting this assessment for marking, I affirm that this assessment is my own work.

Learner Signature

Maram Alkhenaizi

Do not write beyond this line. For assessor use only.

Assessor's Name			
Marking Date		Marks Obtained	

Comments:

Table of Contents

1. Problem Definition, Objectives & Background	5
1.1 Problem Statement.....	5
1.2 Literature Review	5
1.3 Project Objectives	5
1.4 Experimental Setup	5
2. Proposed ML Framework with Data Acquisition & Processing	7
2.1 Proposed Framework	7
2.2 Choice of Dataset	8
2.3 Dataset Description & Statistics	8
2.4 Pre-processing & Data Transformation	9
2.4.1 Drop irrelevant ID columns.....	9
2.4.2 Handle datetime features	10
2.4.3 Encode categorical variables.....	10
2.5 Train-Test Split & Class Imbalance Handling	11
2.5.1 Dataset Split	11
2.5.2 Handle Imbalance (SMOTE)	12
2.5.3 Feature Scaling	12
3. Feature Selection, Learning Methods, Evaluation Strategies & Metrics.....	14
3.1 Feature Selection.....	14
3.2 Learning Methods	15
3.2.1 Logistic Regression	15
3.2.2 Naive Bayes	15
3.2.3 Random Forest	15
3.3 Evaluation Methods	16
3.4 Evaluation Metrics	16
4. Hyper-parameter Optimization & Summarization of Results & Inference	17
4.1 Hyper-parameter Tuning Approach.....	17
4.2 Tune Models	17
4.2.1 Tune Logistic Regression	17
4.2.2 Tune Naive Bayes	18
4.2.3 Tune Random Forest	18

4.3 Tune Models Evaluation.....	19
4.4 Results Summary.....	20
4.5 Inference.....	20
5. Discussions and Future Extensions to the Work	20
5.1 Critical Analysis and Inferences	20
5.2 Comparison with Previous Work	21
5.3 Future Work.....	21
6. Project Migration to a new tool/technology	22
7. GitHub Repository	23
8. Use of Gen AI	24
9. Log Files	25
10. References.....	26

Table of Figures

Figure 1 - ML Framework	7
Figure 2 - Dataset Sample	8
Figure 3 - Dataset Description	8
Figure 4 - Dataset Statistics	9
Figure 5 - Class distribution	9
Figure 6 - Dropping high-cardinality ID columns code	9
Figure 7 - Dropping high-cardinality ID columns output	10
Figure 8 - Datetime feature extraction code	10
Figure 9- Datetime feature extraction output	10
Figure 10 - Encode categorical variables code	11
Figure 11 - Encode categorical variables output	11
Figure 12 - Dataset split code	11
Figure 13 - Dataset split output	12
Figure 14- SMOTE code	12
Figure 15 - Class distribution after SMOTE oversampling	12
Figure 16 - Feature scaling code	12
Figure 17 - Feature scaling output	13
Figure 18 – Random Forest top-15 feature importance code	14
Figure 19 – Random Forest top-15 feature importance bar chart	14
Figure 20 - Logistic Regression training code	15
Figure 21 - Naive Bayes training code	15
Figure 22- Random Forest training code	15
Figure 23 – Models' evaluation metrics output	16
Figure 24 - Models' confusion matrix output	16
Figure 25 - Tune Logistic Regression training code	17
Figure 26 - Tune Logistic Regression training output	18
Figure 27 - Tune Naive Bayes training code	18
Figure 28 - Tune Naive Bayes output	18
Figure 29 - Tune Random Forest training code	19
Figure 30 - Tune Random Forest training output	19
Figure 31 - Tuned models' evaluation metrics output	20
Figure 32 - Tuned models' confusion matrix output	20
Figure 33 - Orange workflow for fraud detection	22

1. Problem Definition, Objectives & Background

1.1 Problem Statement

E-commerce fraud causes billions in annual losses, yet detecting it is difficult because fraudulent transactions account for only 1–5% of the total transactions. This project addresses the challenge of building a machine learning model that accurately classifies transactions as legitimate or fraudulent in real time, despite severe class imbalance and the need for fast, scalable detection.

1.2 Literature Review

Fraud detection methods have developed through three main generations. The first generation used rule-based systems such as velocity rules and blacklists, which were simple but too rigid. The second generation introduced classical machine learning algorithms like Logistic Regression, Naïve Bayes, and Decision Trees, which capture non-linear patterns but require careful feature engineering. The third generation includes deep learning and ensemble methods such as Random Forest, Gradient Boosting, and Autoencoders, which dominate recent fraud detection research (Al Ali et al., 2023; Alazab et al., 2021).

Ensemble methods such as Random Forest have been shown to outperform many other classifiers for e-commerce fraud detection (Amini and Rabiei, 2022), while combining SMOTE with classical and ensemble classifiers significantly improves performance on highly imbalanced datasets (Cheah et al., 2023). This study adopts a comparable pipeline using Logistic Regression, Naïve Bayes, and Random Forest.

1.3 Project Objectives

- Build an efficient e-commerce fraud detection model using a public dataset.
- Apply pre-processing techniques including encoding, scaling, and SMOTE (Synthetic Minority Over-sampling Technique) to handle class imbalance.
- Train and compare three machine learning models.
- Evaluate models using Accuracy, Precision, Recall, F1-Score, ROC-AUC, and Confusion Matrix.
- Recommend the best-performing model and outline avenues for future enhancement.

1.4 Experimental Setup

Component	Specification
Programming Language	Python 3.11
Primary IDE	Jupyter Notebook (Anaconda distribution)
Secondary Tool	Orange Data Mining 3.36 (no-code)
Core Libraries	pandas, NumPy, scikit-learn, imbalanced-learn

Visualisation	matplotlib, seaborn
---------------	---------------------

2. Proposed ML Framework with Data Acquisition & Processing

2.1 Proposed Framework

The framework spans data acquisition, pre-processing, SMOTE-based class balancing, feature scaling, model training and tuning, and final evaluation on the original imbalanced test set.

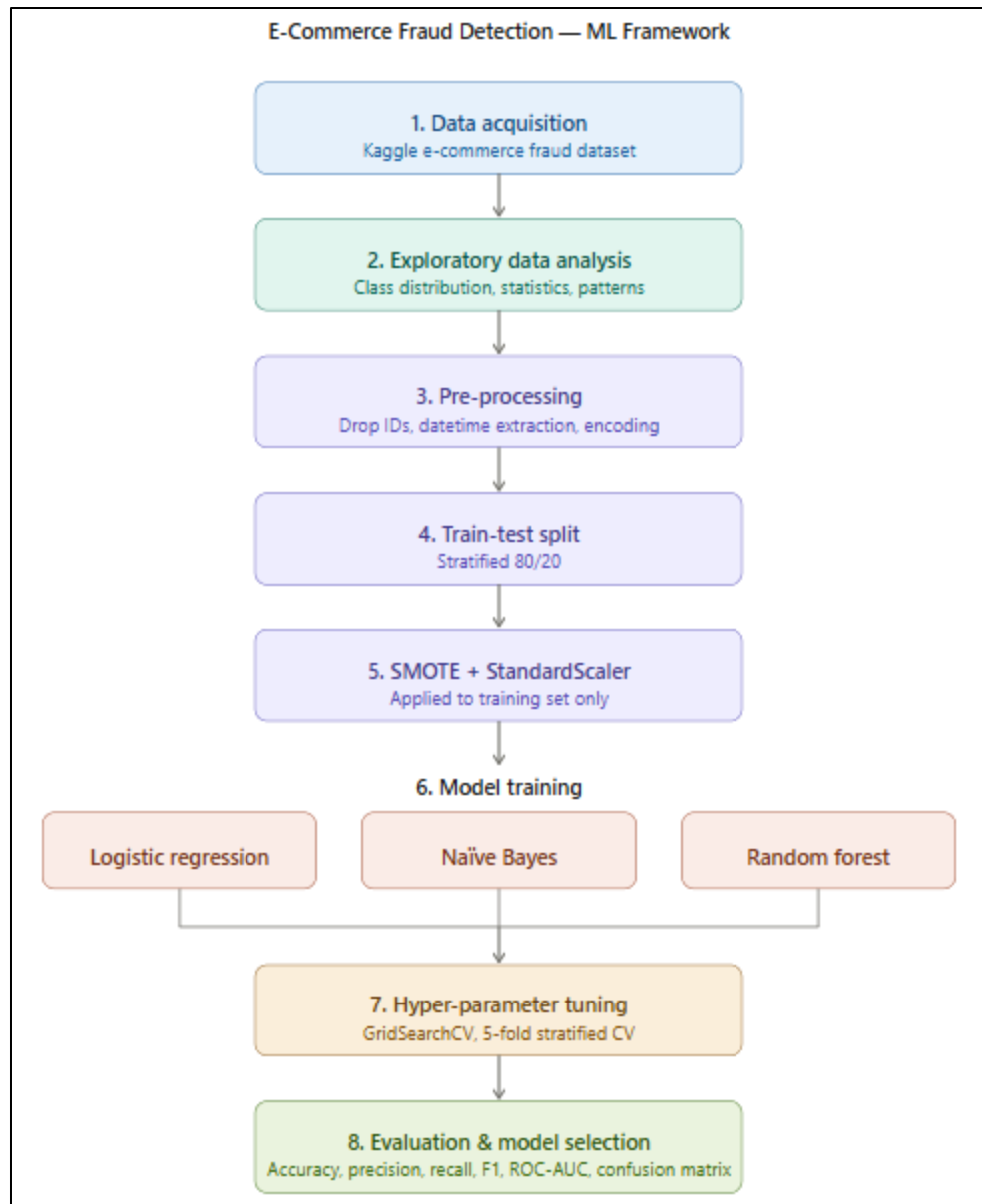


Figure 1 - ML Framework

2.2 Choice of Dataset

The E-Commerce Fraud Detection Dataset from Kaggle (Tuygur, 2025) was selected for its realistic transaction patterns, mix of numerical and categorical fraud-relevant features, and clear binary fraud label.

Dataset Link: <https://www.kaggle.com/datasets/umuttuygurr/e-commerce-fraud-detection-dataset>

A sample of the dataset is shown below, with the label highlighted in **yellow**:

transaction_id	user_id	account_age_days	total_transactions_user	avg_amount_user	amount	country	bin_country	channel	merchant_category	promo_used	avs_match	cvv_result	three_ds_flag	transaction_time	shipping_distance_km	is_fraud
1	1	141	47	147.93	84.75	FR	FR	web	travel	0	1	1	1	2024-01-06T04:09:38Z	370.95	0
2	1	141	47	147.93	107.90	FR	FR	web	travel	0	0	0	0	2024-01-09T20:13:47Z	149.62	0
3	1	141	47	147.93	92.36	FR	FR	app	travel	1	1	1	1	2024-01-12T06:20:11Z	164.08	0
4	1	141	47	147.93	112.47	FR	FR	web	fashion	0	1	1	1	2024-01-15T17:00:04Z	397.40	0
5	1	141	47	147.93	132.91	FR	US	web	electronics	0	1	1	1	2024-01-17T01:27:31Z	935.28	0

Figure 2 - Dataset Sample

2.3 Dataset Description & Statistics

The dataset contains **299,695 rows** and **17 columns** with no missing values.

```
=====
DATASET INFORMATION
=====
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 299695 entries, 0 to 299694
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   transaction_id                        299695 non-null int64
1   user_id                              299695 non-null int64
2   account_age_days                     299695 non-null int64
3   total_transactions_user               299695 non-null int64
4   avg_amount_user                      299695 non-null float64
5   amount                              299695 non-null float64
6   country                             299695 non-null object
7   bin_country                         299695 non-null object
8   channel                             299695 non-null object
9   merchant_category                   299695 non-null object
10  promo_used                          299695 non-null int64
11  avs_match                           299695 non-null int64
12  cvv_result                          299695 non-null int64
13  three_ds_flag                       299695 non-null int64
14  transaction_time                     299695 non-null object
15  shipping_distance_km                 299695 non-null float64
16  is_fraud                            299695 non-null int64
dtypes: float64(3), int64(9), object(5)
```

Figure 3 - Dataset Description

The statistical summary shows wide variation in transaction amounts (mean 177, max 169,94) and account age (1–1890 days), while the fraud rate of 2.2% confirms the dataset's strong class imbalance.

	transaction_id	user_id	account_age_days	total_transactions_user	avg_amount_user	amount	promo_used	avs_match	cvv_result	three_ds_flag	shipping_distance_km	is_fraud
count	299695.0000	299695.000000	299695.000000	299695.000000	299695.000000	299695.000000	299695.000000	299695.000000	299695.000000	299695.000000	299695.000000	299695.000000
mean	149848.0000	3002.559432	973.397871	50.673321	148.142973	177.165279	0.153640	0.837999	0.872110	0.784588	357.049028	0.022062
std	86514.6388	1732.309663	525.241409	5.976391	200.364624	306.926507	0.360603	0.368453	0.333968	0.411109	427.672074	0.146887
min	1.0000	1.000000	1.000000	40.000000	3.520000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	74924.5000	1501.000000	516.000000	46.000000	46.190000	42.100000	0.000000	1.000000	1.000000	1.000000	136.600000	0.000000
50%	149848.0000	3007.000000	975.000000	51.000000	90.130000	89.990000	0.000000	1.000000	1.000000	1.000000	273.020000	0.000000
75%	224771.5000	4504.000000	1425.000000	56.000000	173.450000	191.110000	0.000000	1.000000	1.000000	1.000000	409.180000	0.000000
max	299695.0000	6000.000000	1890.000000	60.000000	4565.290000	16994.740000	1.000000	1.000000	1.000000	1.000000	3748.560000	1.000000

Figure 4 - Dataset Statistics

The class distribution confirms the severe imbalance as well.

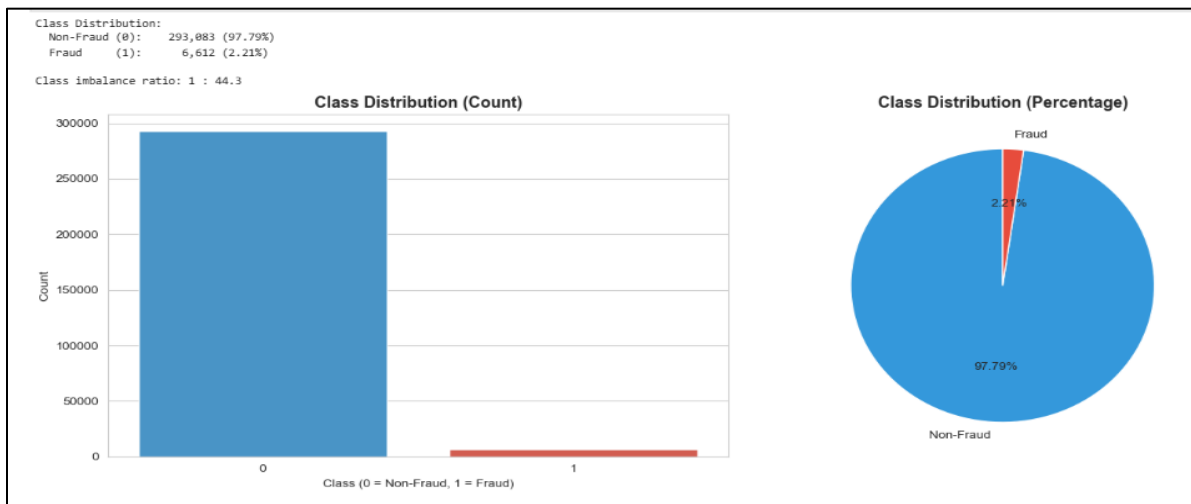


Figure 5 - Class distribution

2.4 Pre-processing & Data Transformation

2.4.1 Drop irrelevant ID columns

ID columns have high cardinality and no predictive value, so they were removed. Dropping **transaction_id** reduced the dataset to 16 columns.

```
# Make a copy
df_processed = df.copy()

# Step 1: Drop ID-type columns (no predictive value)
id_like_cols = []
for col in df_processed.columns:
    if 'id' in col.lower() and col != TARGET_COL:
        if df_processed[col].nunique() > 0.95 * len(df_processed): # Very high cardinality
            id_like_cols.append(col)

print(f"Dropping ID-like columns: {id_like_cols}")
df_processed = df_processed.drop(columns=id_like_cols)
print(f"Shape after dropping IDs: {df_processed.shape}")
```

Figure 6 - Dropping high-cardinality ID columns code

```
Dropping ID-like columns: ['transaction_id']
Shape after dropping IDs: (299695, 16)
```

Figure 7 - Dropping high-cardinality ID columns output

2.4.2 Handle datetime features

The **transaction_time** field is split into hour, day, month, and day-of-week features, capturing temporal patterns relevant to fraud.

```
# Step 2: Handle datetime columns

# Show columns BEFORE processing
print("\n" * 60)
print("COLUMNS BEFORE DATETIME PROCESSING")
print("\n" * 60)
print(f"Total columns: {df_processed.shape[1]}")
print(list(df_processed.columns))

# Detect datetime columns
datetime_cols = []
for col in df_processed.columns:
    if df_processed[col].dtype == 'object':
        try:
            sample = df_processed[col].dropna().iloc[0]
            pd.to_datetime(sample)
            if '-' in str(sample) or '/' in str(sample) or ':' in str(sample):
                datetime_cols.append(col)
        except:
            continue
    elif 'datetime' in str(df_processed[col].dtype):
        datetime_cols.append(col)

# Extract features
for col in datetime_cols:
    try:
        df_processed[col] = pd.to_datetime(df_processed[col], errors='coerce')
        df_processed[f'{col}_hour'] = df_processed[col].dt.hour
        df_processed[f'{col}_day'] = df_processed[col].dt.day
        df_processed[f'{col}_month'] = df_processed[col].dt.month
        df_processed[f'{col}_dayofweek'] = df_processed[col].dt.dayofweek
        df_processed = df_processed.drop(columns=[col])
        print(f"Extracted hour/day/month/dayofweek from '{col}'")
    except Exception as e:
        print(f"Could not process {col}: {e}")

# Show columns AFTER processing
print("\n" * 60)
print("COLUMNS AFTER DATETIME PROCESSING")
print("\n" * 60)
print(f"Total columns: {df_processed.shape[1]}")
print(list(df_processed.columns))
print(f"Shape after datetime processing: {df_processed.shape}")
```

Figure 8 - Datetime feature extraction code

The dataset expanded from 16 to 19 columns after extraction.

```
=====
COLUMNS BEFORE DATETIME PROCESSING
=====
Total columns: 16
['user_id', 'account_age_days', 'total_transactions_user', 'avg_amount_user', 'amount', 'country', 'bin_country', 'channel', 'merchant_category', 'promo_used', 'avs_match', 'cvv_result', 'three_ds_flag', 'transaction_time', 'shipping_distance_kn', 'is_fraud']
Extracted hour/day/month/dayofweek from 'transaction_time'

=====
COLUMNS AFTER DATETIME PROCESSING
=====
Total columns: 19
['user_id', 'account_age_days', 'total_transactions_user', 'avg_amount_user', 'amount', 'country', 'bin_country', 'channel', 'merchant_category', 'promo_used', 'avs_match', 'cvv_result', 'three_ds_flag', 'shipping_distance_kn', 'is_fraud', 'transaction_time_hour', 'transaction_time_day', 'transaction_time_month', 'transaction_time_dayofweek']
Shape after datetime processing: (299695, 19)
```

Figure 9- Datetime feature extraction output

2.4.3 Encode categorical variables

All four categorical columns were label-encoded, leaving the dataset fully numeric.

```

# Step 3: Encode categorical variables

# Show categorical columns BEFORE encoding
categorical_cols = df_processed.select_dtypes(include=['object']).columns.tolist()
print("\n" * 60)
print("CATEGORICAL COLUMNS BEFORE ENCODING")
print("\n" * 60)
print(f"Total categorical columns: {len(categorical_cols)}")
print(categorical_cols)

# Label encode all categorical columns
print("\n" * 60)
print("LABEL ENCODING")
print("\n" * 60)
label_encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    df_processed[col] = le.fit_transform(df_processed[col].astype(str))
    label_encoders[col] = le
    print(f"Encoded '{col}' -> {len(le.classes_)} categories")

# Show columns AFTER encoding
print("\n" * 60)
print("CATEGORICAL COLUMNS AFTER ENCODING")
print("\n" * 60)
remaining_obj = df_processed.select_dtypes(include=['object']).columns.tolist()
print(f"Remaining object-type columns: {len(remaining_obj)}")
print(f"All columns are now numeric: {len(remaining_obj) == 0}")
print(f"Final shape: {df_processed.shape}")

```

Figure 10 - Encode categorical variables code

```

=====
CATEGORICAL COLUMNS BEFORE ENCODING
=====
Total categorical columns: 4
['country', 'bin_country', 'channel', 'merchant_category']

=====
LABEL ENCODING
=====
Encoded 'country' -> 10 categories
Encoded 'bin_country' -> 10 categories
Encoded 'channel' -> 2 categories
Encoded 'merchant_category' -> 5 categories

=====
CATEGORICAL COLUMNS AFTER ENCODING
=====
Remaining object-type columns: 0
All columns are now numeric: True

Final shape: (299695, 19)

```

Figure 11 - Encode categorical variables output

2.5 Train-Test Split & Class Imbalance Handling

2.5.1 Dataset Split

Split the data into training and test sets is performed before SMOTE and scaling to prevent data leakage. The 80/20 split balances sufficient training data with a reliable test set.

```

# Separate features and target
X = df_processed.drop(columns=[TARGET_COL])
y = df_processed[TARGET_COL]

print(f"Features (X): {X.shape}")
print(f"Target (y) : {y.shape}")

# Stratified train-test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)

print(f"\nTraining set: {X_train.shape[0]:,} rows")
print(f"Testing set : {X_test.shape[0]:,} rows")
print(f"\nFraud rate in train: {y_train.mean()*100:.2f}%")
print(f"\nFraud rate in test : {y_test.mean()*100:.2f}%")

```

Figure 12 - Dataset split code

```
Features (X): (299695, 18)
Target (y) : (299695,)

Training set: 239,756 rows
Testing set : 59,939 rows

Fraud rate in train: 2.21%
Fraud rate in test : 2.21%
```

Figure 13 - Dataset split output

2.5.2 Handle Imbalance (SMOTE)

SMOTE oversamples the minority class, creating a perfectly balanced training set with 234,466 samples per class.

```
# Apply SMOTE to handle class imbalance (training set only)
print("Applying SMOTE to balance the training set")
smote = SMOTE(random_state=RANDOM_STATE)
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

print(f"\nBefore SMOTE:")
print(f"  Class 0 (Non-Fraud): {(y_train==0).sum():,}")
print(f"  Class 1 (Fraud)    : {(y_train==1).sum():,}")
print(f"\nAfter SMOTE:")
print(f"  Class 0 (Non-Fraud): {(y_train_smote==0).sum():,}")
print(f"  Class 1 (Fraud)    : {(y_train_smote==1).sum():,}")
print(f"\nClasses are now balanced")
```

Figure 14- SMOTE code

As shown in the bar and donut chart below, the classes are now balanced.

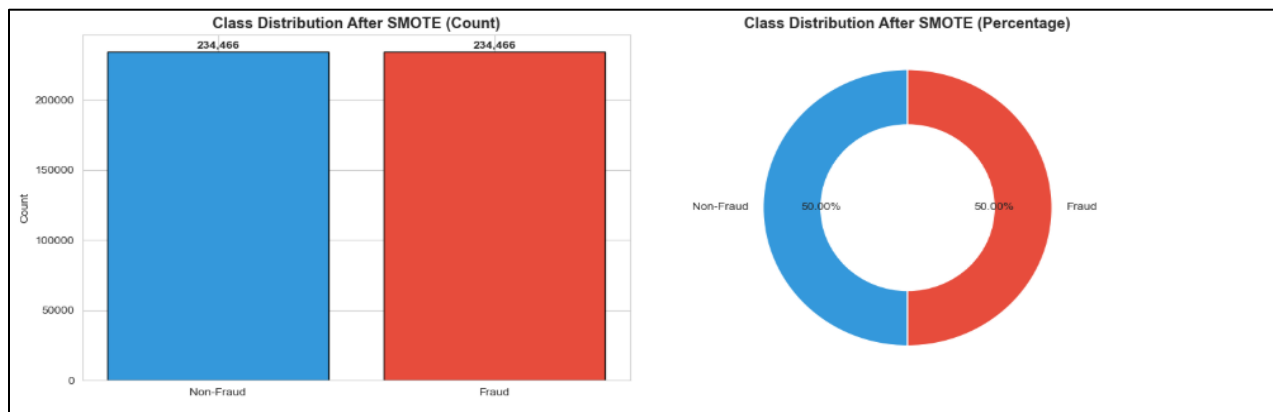


Figure 15 - Class distribution after SMOTE oversampling

2.5.3 Feature Scaling

Features are scaled using StandardScaler to ensure consistent feature ranges.

```
# Feature scaling (essential for Logistic Regression)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_smote)
X_test_scaled = scaler.transform(X_test)

print(" Features scaled using StandardScaler")
print(f"  Mean after scaling: {X_train_scaled.mean():.4f} (should be ~0)")
print(f"  Std after scaling : {X_train_scaled.std():.4f} (should be ~1)")
```

Figure 16 - Feature scaling code

```
Features scaled using StandardScaler  
Mean after scaling: 0.0000  
Std after scaling : 1.0000
```

Figure 17 - Feature scaling output

3. Feature Selection, Learning Methods, Evaluation Strategies & Metrics

3.1 Feature Selection

Feature selection was performed through two complementary mechanisms which are high-cardinality identifier columns were dropped during pre-processing, and Random Forest's built-in feature importance ranked features by gain, providing model-driven evidence of feature relevance. This combined filter-and-embedded approach offers both interpretability and empirical justification for the retained features.

```
# Feature Importance from Random Forest
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf_model.feature_importances_
}).sort_values('Importance', ascending=False)

plt.figure(figsize=(10, 6))
top_features = feature_importance.head(15)
sns.barplot(data=top_features, x='Importance', y='Feature', palette='viridis')
plt.title('Top 15 Most Important Features (Random Forest)', fontweight='bold')
plt.xlabel('Feature Importance')
plt.tight_layout()
plt.show()

print("\nTop 10 features:")
print(feature_importance.head(10).to_string(index=False))
```

Figure 18 – Random Forest top-15 feature importance code

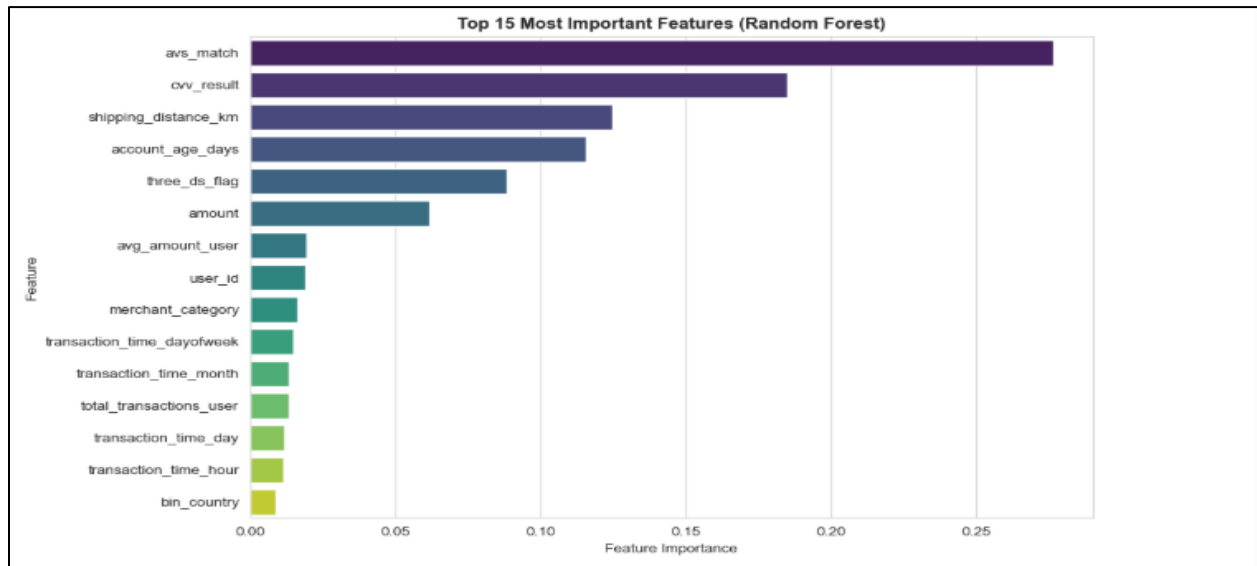


Figure 19 – Random Forest top-15 feature importance bar chart

3.2 Learning Methods

3.2.1 Logistic Regression

Logistic Regression was chosen as a simple, interpretable baseline. Its coefficients directly show how each feature influences fraud probability.

```
# Train Logistic Regression
print("Training Logistic Regression model")
start_time = time.time()

lr_model = LogisticRegression(
    max_iter=1000,
    random_state=RANDOM_STATE,
    n_jobs=-1
)
lr_model.fit(X_train_scaled, y_train_smote)

lr_train_time = time.time() - start_time
print(f" Logistic Regression trained in {lr_train_time:.2f} seconds")
```

Figure 20 - Logistic Regression training code

3.2.2 Naive Bayes

Naive Bayes was chosen as a fast probabilistic baseline based on Bayes' theorem, offering a different methodological paradigm compared to linear and tree-based models.

```
# Train Naive Bayes
from sklearn.naive_bayes import GaussianNB

print("Training Naive Bayes model")
start_time = time.time()

nb_model = GaussianNB()
nb_model.fit(X_train_scaled, y_train_smote)

nb_train_time = time.time() - start_time
print(f" Naive Bayes trained in {nb_train_time:.2f} seconds")
```

Figure 21 - Naive Bayes training code

3.2.3 Random Forest

Random Forest was chosen as an ensemble method that combines many decision trees to capture complex patterns while reducing overfitting.

```
# Train Random Forest

print("Training Random Forest model")
start_time = time.time()

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=RANDOM_STATE,
    n_jobs=-1
)
rf_model.fit(X_train_smote, y_train_smote)

rf_train_time = time.time() - start_time
print(f" Random Forest trained in {rf_train_time:.2f} seconds")
```

Figure 22- Random Forest training code

3.3 Evaluation Methods

The evaluation methodology relied on a stratified 80/20 train-test split (**Section 2.5.1**), with SMOTE and StandardScaler applied only to the training set (**Sections 2.5.2–2.5.3**). The test set was held out entirely to provide an unbiased measure of generalization performance.

3.4 Evaluation Metrics

Five metrics and a confusion matrix were computed to evaluate model performance on imbalanced classification:

- **Accuracy:** overall correctness, interpreted cautiously due to the 1:44 imbalance.
- **Precision:** fraction of predicted fraud that is truly fraud; reduces false alerts.
- **Recall:** fraction of real fraud detected; reduces financial losses.
- **F1-Score:** harmonic mean of precision and recall.
- **ROC-AUC:** discrimination ability across thresholds, robust to imbalance.
- **Confusion Metrics:** breakdown of true positives, true negatives, false positives, and false negatives, providing a complete view of model errors.

These were **evaluated** on the held-out test set; results are shown below.

MODEL EVALUATION METRICS					
	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.9188	0.1837	0.7791	0.2973	0.9262
Naïve Bayes	0.9185	0.1800	0.7587	0.2910	0.9145
Random Forest	0.9815	0.5573	0.7799	0.6501	0.9671

Figure 23 – Models' evaluation metrics output

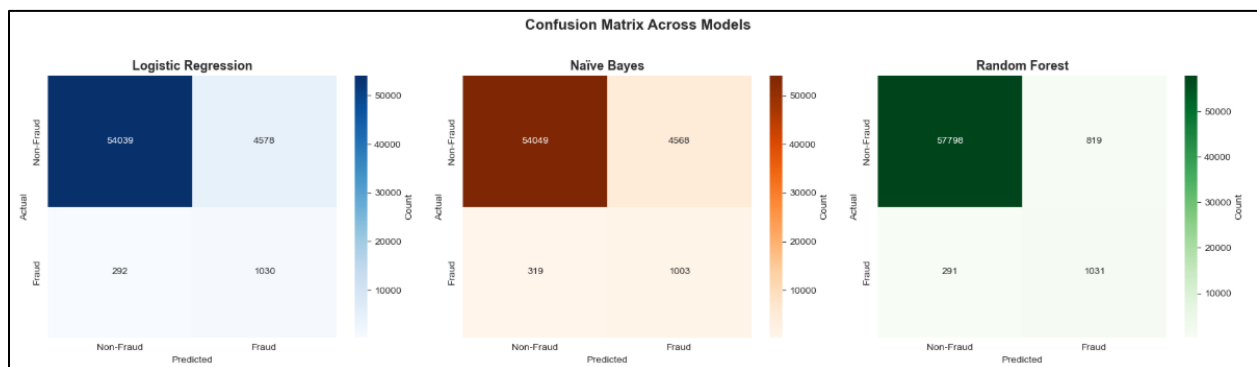


Figure 24 - Models' confusion matrix output

4. Hyper-parameter Optimization & Summarization of Results & Inference

4.1 Hyper-parameter Tuning Approach

GridSearchCV with 5-fold cross-validation was applied to all three models. GridSearchCV requires a single optimization metric to rank parameter combinations, and F1-score was selected because it balances precision and recall in our imbalanced setting.

4.2 Tune Models

4.2.1 Tune Logistic Regression

GridSearchCV with **5-fold cross-validation** tuned **C** and **solver** for Logistic Regression, identifying **C=0.01** and **solver='liblinear'** as the optimal combination with a CV F1-score of **0.9257**.

```
# --- Hyper-parameter tuning for Logistic Regression ---
param_grid_lr = {
    'C'      : [0.01, 0.1, 1, 10],
    'solver': ['lbfgs', 'liblinear']
}

print("Tuning Logistic Regression")
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

grid_lr = GridSearchCV(
    estimator=LogisticRegression(max_iter=1000, random_state=RANDOM_STATE),
    param_grid=param_grid_lr,
    scoring='f1',
    cv=cv,
    n_jobs=-1,
    verbose=1
)

start = time.time()
grid_lr.fit(X_train_scaled, y_train_smote)
print(f"\n Completed in {(time.time()-start)/60:.1f} minutes")
print(f"Best parameters: {grid_lr.best_params_}")
print(f"Best CV F1-Score: {grid_lr.best_score_:.4f}")

lr_tuned = grid_lr.best_estimator_
```

Figure 25 - Tune Logistic Regression training code

```
Tuning Logistic Regression
Fitting 5 folds for each of 8 candidates, totalling 40 fits

Completed in 0.9 minutes
Best parameters: {'C': 0.01, 'solver': 'liblinear'}
Best CV F1-Score: 0.9257
```

Figure 26 - Tune Logistic Regression training output

4.2.2 Tune Naive Bayes

GridSearchCV with 5-fold cross-validation tuned `var_smoothing` for Naïve Bayes, identifying `1e-12` as the optimal value with a CV F1-score of **0.9221**.

```
# --- Hyper-parameter tuning for Naïve Bayes ---
param_grid_nb = {
    'var_smoothing': [1e-12, 1e-10, 1e-9, 1e-8, 1e-7, 1e-6]
}

print("Tuning Naïve Bayes")
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

grid_nb = GridSearchCV(
    estimator=GaussianNB(),
    param_grid=param_grid_nb,
    scoring='f1',
    cv=cv,
    n_jobs=-1,
    verbose=1
)

start = time.time()
grid_nb.fit(X_train_scaled, y_train_smote)
print(f"\n Completed in {(time.time()-start)/60:.1f} minutes")
print(f"Best parameters: {grid_nb.best_params_}")
print(f"Best CV F1-score: {grid_nb.best_score_:.4f}")

nb_tuned = grid_nb.best_estimator_
```

Figure 27 - Tune Naive Bayes training code

```
Tuning Naïve Bayes
Fitting 5 folds for each of 6 candidates, totalling 30 fits

Completed in 0.2 minutes
Best parameters: {'var_smoothing': 1e-12}
Best CV F1-score: 0.9221
```

Figure 28 - Tune Naive Bayes output

4.2.3 Tune Random Forest

GridSearchCV with 5-fold stratified cross-validation tuned `n_estimators`, identifying 100 as the optimal value with a CV F1-score of **0.9356**.

```

# --- Hyper-parameter tuning for Random Forest ---

# Subsample 48%
X_train_subsample, y_train_subsample = resample(
    X_train_smote, y_train_smote,
    n_samples=int(len(X_train_smote) * 0.4),
    random_state=RANDOM_STATE,
    stratify=y_train_smote
)

print(f"Original training set: {len(X_train_smote):,} samples")
print(f"Subsampled training set: {len(X_train_subsample):,} samples (48%)")

param_grid_rf = (
    'n_estimators': [50, 100]
)

print(f"\nTuning Random Forest")
print(f"Total combinations: 2")
print(f"With 5-fold stratified CV: 10 model fits")

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

grid_rf = GridSearchCV(
    estimator=RandomForestClassifier(random_state=RANDOM_STATE, n_jobs=-1),
    param_grid=param_grid_rf,
    scoring='f1',
    cv=cv,
    n_jobs=-1,
    verbose=2
)

start = time.time()
grid_rf.fit(X_train_subsample, y_train_subsample)
print(f"\nCompleted in {(time.time()-start)/60:.1f} minutes")
print(f"Best parameters: {grid_rf.best_params_}")
print(f"Best CV F1-score: {grid_rf.best_score_:0.4f}")

# Re-train the FINAL model on FULL training data with best parameters
print("\nRe-training the final model on the full training data")
rf_tuned = RandomForestClassifier(
    **grid_rf.best_params_,
    random_state=RANDOM_STATE,
    n_jobs=-1
)
rf_tuned.fit(X_train_smote, y_train_smote)
print("Final tuned Random Forest trained on full data!")

```

Figure 29 - Tune Random Forest training code

```

Original training set: 468,932 samples
Subsampled training set: 187,572 samples (48%)

Tuning Random Forest
Total combinations: 2
With 5-fold stratified CV: 10 model fits
Fitting 5 folds for each of 2 candidates, totalling 10 fits

Completed in 6.3 minutes
Best parameters: {'n_estimators': 100}
Best CV F1-score: 0.9856

Re-training the final model on the full training data
Final tuned Random Forest trained on full data!

```

Figure 30 - Tune Random Forest training output

4.3 Tune Models Evaluation

Each tuned model was evaluated on the held-out test set using the below five metrics and confusion matrix.

MODEL COMPARISON – ALL TUNED MODELS					
	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression (Tuned)	0.9188	0.1838	0.7791	0.2974	0.9260
Naïve Bayes (Tuned)	0.9185	0.1800	0.7587	0.2910	0.9145
Random Forest (Tuned)	0.9815	0.5573	0.7799	0.6501	0.9671

BEST MODEL PER METRIC					
Accuracy	:	Random Forest (Tuned)	(0.9815)		
Precision	:	Random Forest (Tuned)	(0.5573)		
Recall	:	Random Forest (Tuned)	(0.7799)		
F1-Score	:	Random Forest (Tuned)	(0.6501)		
ROC-AUC	:	Random Forest (Tuned)	(0.9671)		

Figure 31 - Tuned models' evaluation metrics output

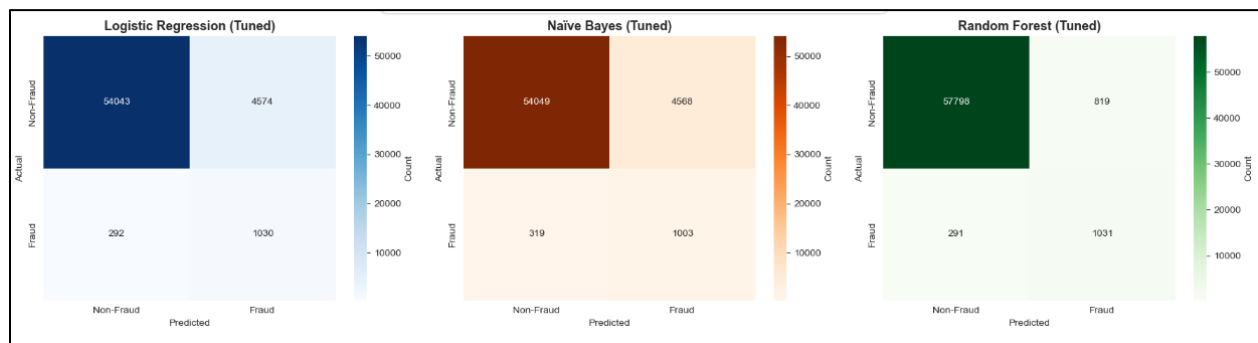


Figure 32 - Tuned models' confusion matrix output

4.4 Results Summary

For fraud detection, F1-score and Recall are the most critical metrics. **Random Forest** is the best performer with **F1: 0.65** and **Recall: 0.78**, with only **819** false positives. **Logistic Regression** follows with **F1: 0.30** and **Recall: 0.78**, catching **1,030** fraud cases but producing **4,579** false positives. **Naïve Bayes** is the worst with **F1: 0.29** and **Recall: 0.76**, with similarly high false positives.

4.5 Inference

Random Forest (Tuned) is the recommended model for deployment. It captures complex feature interactions, balances precision and recall well, and outperforms the other models clearly. Hyper-parameter tuning helped Random Forest the most, while Logistic Regression and Naïve Bayes showed little improvement.

5. Discussions and Future Extensions to the Work

5.1 Critical Analysis and Inferences

The results highlight the strengths and limitations of each model. **Random Forest** performed best because it captures complex non-linear feature interactions in fraud data, balancing precision and recall effectively. **Logistic Regression** and **Naïve Bayes** achieved high recall but very low precision, generating excessive false positives that would burden customer service in deployment. Hyper-parameter tuning produced clear

gains only for Random Forest, indicating that ensemble methods benefit most from optimization on imbalanced tabular fraud datasets.

5.2 Comparison with Previous Work

The findings align with recent fraud detection research. Consistent with Amini and Rabiei (2022), ensemble methods such as Random Forest outperformed linear and probabilistic classifiers. The improvement from combining SMOTE with classical and ensemble classifiers — also observed by Cheah et al. (2023) — was confirmed in our results. Unlike Al Ali et al. (2023) and Alazab et al. (2021), which use deep learning, this project prioritized interpretable classical models for transparency and reproducibility.

5.3 Future Work

Future extensions include applying reinforcement learning to dynamically adapt the fraud detection policy as new transaction patterns emerge. Other directions include experimenting with deep learning models such as LSTM or autoencoders for sequential behavior, integrating real-time streaming data via Apache Kafka, and incorporating user-level features such as historical transaction patterns to improve detection accuracy.

6. Project Migration to a new tool/technology

To demonstrate cross-tool migration, the same fraud detection pipeline was re-implemented in **Orange Data Mining 3.36**, a no-code visual programming platform. Orange was chosen because it offers a fundamentally different paradigm (visual drag-and-drop versus Python's code-based approach), provides all required components natively (Preprocess, Data Sampler, model widgets, Test and Score, and Confusion Matrix).

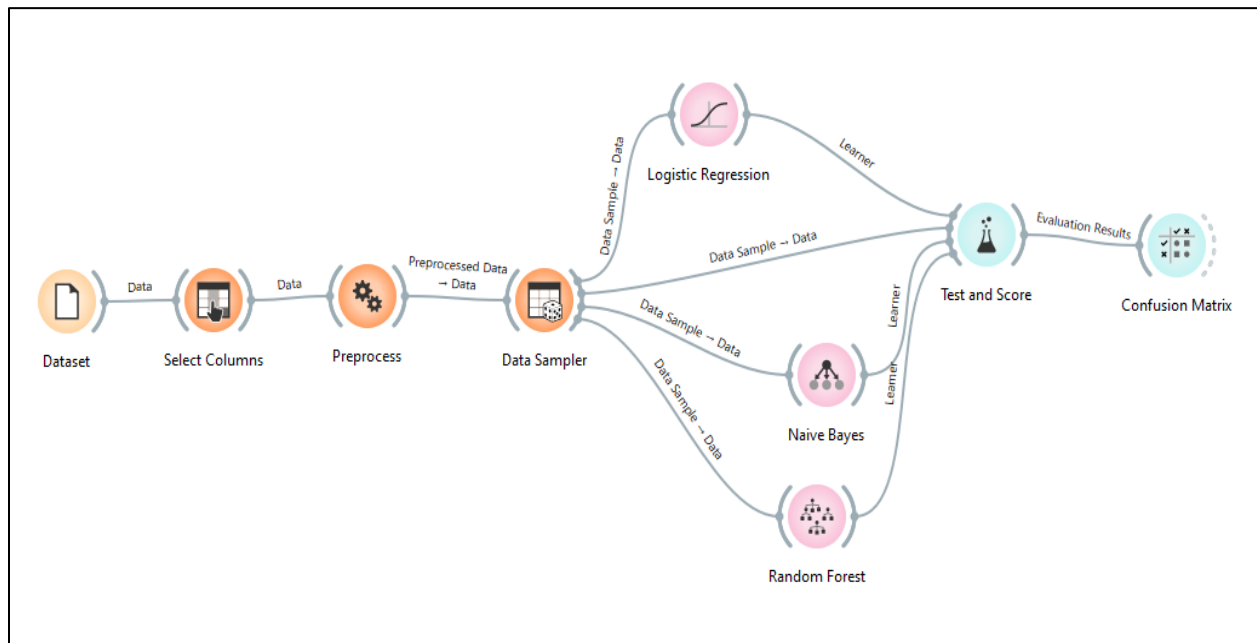


Figure 33 - Orange workflow for fraud detection

7. GitHub Repository

GitHub Link: <https://github.com/maramkhunaizi/E-Commerce-Fraud-Detection>

8. Use of Gen AI

Generative AI (Claude by Anthropic) was used as a supportive tool to:

1. Refine report wording for clarity and academic tone.
2. Explain ML concepts such as SMOTE and GridSearchCV.
3. Suggest code structure for evaluation and visualisation.
4. Produce visual aids like the framework diagram.

Note: All implementation, model evaluation, and critical analysis were performed independently by the author.

9. Log Files

Date	Task	Task Description	Challenges if any
18/Apr/2026	Project briefing & dataset exploration	- Reviewed the project description and rubric in detail. - Explored Kaggle and selected the E-Commerce Fraud Detection Dataset. - Drafted initial project objectives and problem definition.	N/A
25/Apr/2026	Environment setup & initial EDA	- Set up Jupyter Notebook environment with required libraries. - Performed initial Exploratory Data Analysis (EDA).	N/A
2/May/2026	Data pre-processing pipeline	- Dropped high-cardinality ID columns. - Extracted temporal features (hour, day, month, day-of-week) from transaction_time. - Label-encoded categorical columns.	N/A
9/May/2026	Train-test split & class balancing	- Performed 80/20 train-test split. - Applied SMOTE and StandardScaler to the training set only to prevent data leakage.	N/A
16/May/2026	Model implementation in Python	- Implemented Logistic Regression. - Implemented Naïve Bayes. - Implemented Random Forest. - Evaluated all three models using five metrics (Accuracy, Precision, Recall, F1-Score, ROC-AUC) and confusion matrices.	N/A
23/May/2026	Hyper-parameter tuning	- Applied GridSearchCV with 5-fold stratified cross-validation to all three models using F1-score as the scoring metric.	N/A
31/May/2026	Orange migration, critical analysis & GitHub upload	- Re-implemented all three models in Orange for the project migration to a new tool component, using the pre-processed dataset. - Compared default vs tuned configurations and produced final model comparison table and confusion matrices. - Created the project GitHub repository. - Finalized the report and references.	N/A

10. References

Al Ali, A., Alazab, M., & Khan, S. (2023). A hybrid deep learning model for financial fraud detection using blockchain and ensemble methods. *Computers*, 12(3), 78. <https://doi.org/10.3390/computers12030078>

Alazab, M., Tang, M., & Alazab, M. (2021). Deep learning for cybersecurity and fraud detection in financial transactions. *Electronics*, 10(5), 593. <https://doi.org/10.3390/electronics10050593>

Amini, M., & Rabiei, M. (2022). Ensemble learning for fraud detection in e-commerce transactions: A comparative study. *Journal of Applied Intelligent Systems & Information Sciences*.

Anthropic. (2026). *Claude (Opus 4.7) [Large language model]*. <https://claude.ai>

Cheah, P. C. Y., Yang, Y., & Lee, B. G. (2023). Enhancing financial fraud detection through addressing class imbalance using hybrid SMOTE-GAN techniques. *International Journal of Financial Studies*, 11(3), 110. <https://doi.org/10.3390/ijfs11030110>

Tuygur, U. (2025). *E-Commerce Fraud Detection Dataset* [Data set]. Kaggle. <https://www.kaggle.com/datasets/umuttuygurr/e-commerce-fraud-detection-dataset>